

广州软件学院
GUANGZHOU UNIVERSITY OF SOFTWARE

实验报告封面

课程名称： Python Web 企业级项目开发 **课程代码：**

SK3017

任课老师： 陈广豪 **实验指导老师:** 陈广豪

实验报告名称： 课程设计

学生姓名：

学号：

教学班：

递交日期：

签收人： 陈广豪

我申明，本报告内的实验已按要求完成，报告完全是由我个人完成，并没有抄袭行为。我已经保留了这份实验报告的副本。

申明人(签名)：

实验报告评语与评分：

评阅老师签名：

一、课程名称：Python Web 企业级项目开发

二、实验日期：

三、实验目的：

本课程设计采用“题库选题 + AI 编程助手协作”的方式，独立完成一个可运行、可演示的完整 Web 项目：从公开数据集下载与清洗、接口开发，一直做到大屏实现和文档整理。通过本设计，学生应能：

1. **掌握用 Flask 写接口和页面**：路由、模板渲染、请求参数处理、返回 JSON，以及用 sqlite3 直接读写数据库；

2. **理解前后端分离页面的加载过程**：页面只负责骨架，数据由 JS 请求接口拿到后再渲染，不在模板里写死；

3. **能把统计放在数据库里做**：会用 GROUP BY、CASE WHEN、COALESCE 写聚合 SQL，知道为什么不能把整张表拉到前端再算；

4. **能用 ECharts 做大屏页面**：多图表布局、深色主题、地图组件、窗口自适应和图表联动；

5. **借助 AI 编程助手完成开发并记录过程**：整理有效提示词，自己读懂并修改生成的代码，把踩过的坑写进 Debug 日志。

实验用的仪器和材料：

硬件：PC 电脑一台，网络正常；

配置：Windows 10/11（64 位），内存 8G 及以上，硬盘留出 500G 空间；

软件：Python 3.11、VS Code、Flask、SQLite（用 DB Browser for SQLite 看数据）、Chrome/Edge 浏览器、ECharts 5.5、Selenium（自动截图）。

四、实验内容与要求：

选题路线：☑ 路线 B 题库选题——题目六 数据可视化大屏，项目名称《全国新能源汽车充电基础设施运营态势大屏》。

项目简介与功能清单：项目是一个全国充电基础设施运营大屏。数据来自公开的充电站数据集，清洗后入 SQLite，Flask 提供 7 个数据接口，前端一个页面用 ECharts 画 5 张图、顶部 6 个指标卡、底部一条站点明细列表；点击中国地图上的省份可以整屏下钻，点“重置下钻”回到全国。主要功能：① 6 个指标卡（充电站总数、快充桩总数、慢充桩总数、覆盖城市数、平均电费、平均服务费）；② 各省充电站数量热力地图；③ 站点规模分布环形图；④ 快充/慢充桩结构玫瑰图；⑤ 城市充电站数量 TOP10；⑥ 充电电费区间分布；⑦ 底部站点明细（每 8 秒抽样刷新）；⑧ 省份下钻联动与一键重置；⑨ 1920×1080 一屏自适应。

一、报告正文：

1、系统架构与数据库设计

一、整体架构

项目仍然是“页面 + Flask 接口 + SQLite”三层：浏览器打开一个页面，页面加载后由 JS 请求 7 个接口，拿回 JSON 交给 ECharts 渲染；所有统计都在 SQL 里算完，接口只返回结果。页面按 1920×1080 固定画布设计，打开时按窗口大小整体等比缩放并居中，所以评阅机不管什么分辨率都不会出现滚动条。

展现层：一个 index.html 加 screen.css、screen_engine.js。深色底 #070c18，卡片半

透明加模糊，ECharts 5 和 china.json 都放在本地 static/js 下，不依赖 CDN。

服务层：Flask 一共 8 个路由——/ 返回页面，/api/kpi、/api/map-data、/api/city-top、/api/scale-dist、/api/pile-structure、/api/price-dist 是 6 个聚合接口，/api/station-feed 给底部明细列表用。除地图接口外都支持 ?province= 过滤，SQL 用 ? 占位符传参。

数据层：SQLite，database/analytics.db，单表 charging_stations，33,652 条记录，31 个省级行政区、633 个城市；建了 province、city、scale_level 三个索引。

二、数据来源与清洗

数据来源：题目要求使用公开数据集。本项目用的是 GitHub 公开仓库 xjtulyc/Electric_Vehicle_Crawler 里 crawler/充电桩/ 目录下的数据（https://github.com/xjtulyc/Electric_Vehicle_Crawler），按省份存成 33 个 xlsx 文件，字段有站点名称、地址、快充数量、慢充数量、支付方式、充电费、服务费、停车费、开放时间。原始一共 34,186 条站点记录。

清洗过程写在 database/clean_dataset.py 里，按顺序做了五件事：① 解析字段——从地址里取省和市，从“充电费/服务费”文本里取分时段价格求均值（原始数据有 5 种写法），从停车费判断是否免费，从开放时间判断是否 24 小时；② 按“名称 + 地址”去重 63 条；③ 快充和慢充都为 0、或单项超过 500 个的异常记录剔除 471 条；④ 电费小于等于 0 或大于 3 元/度的值视为异常置空，共 889 条；⑤ 价格缺失用同省中位数填

充（电费 1,798 条、服务费 7,173 条）。最后入库 33,652 条，每一步的处理条数都写在

docs/cleaning_report.json 里。

表结构（单表，没有拆维度表，因为查询都是单表聚合）：

· id：自增主键；

· station_name：站点名称；

· province / city / address：省、市、详细地址；

· fast_piles / slow_piles / total_piles：快充桩数、慢充桩数、合计；

· electricity_price / service_price / total_price：电费、服务费、合计单价（元/度）；

· parking_free：停车是否免费（1/0）；

· open_24h：是否 24 小时开放（1/0）；

· scale_level：按站点桩数分的规模档位（1-5、6-10、11-20、20 以上）。

索引：idx_province(province)、idx_city(city)、idx_scale(scale_level)。

2、核心功能实现说明

一、数据清洗与入库

清洗脚本里最麻烦的是价格解析。原始数据里充电费有五种写法，第一版正则只覆盖了一

种，导致 80% 的记录解析失败，后来把四种提取方式串起来才解决：

```
# 节选自 database/clean_dataset.py：分时段价格求平均
def parse_avg_price(text):
    s = str(text).replace("充电费：", "").replace("服务费：", "")
    s = re.sub(r"\s+", "", s)
    if not s or any(w in s for w in EMPTY_PRICE_WORDS):
        return None
```

```
    prices = [float(x) for x in TIME_PRICE_RE.findall(s)]          # 00:00-06:00:0.8200
    if not prices:
        prices = [float(x) for x in LABEL_PRICE_RE.findall(s)]   # 电价:0.500
    if not prices:
        pairs = RANGE_PRICE_RE.findall(s)                                  # 0.2775-1.0574 元/度
        prices = [(float(a) + float(b)) / 2 for a, b in pairs]
    if not prices:
        m = re.match(r"^(\d+(?:\.\d+)?)", s)                           # 1.20
        if m:
            prices = [float(m.group(1))]
    return round(sum(prices) / len(prices), 3) if prices else None
```

清洗完用 pandas 写 CSV（utf-8-sig，Excel 打开不乱码），再用 sqlite3 建表入库，整个过程一条命令：python database/clean_dataset.py。

二、接口与 SQL 聚合

6 个聚合接口都是先取 province 参数，拼出 WHERE 子句和参数列表，然后一条 SQL 用 COUNT/SUM/AVG/GROUP BY/CASE WHEN 算完返回。以指标卡为例：

```
# 节选自 app/server.py：顶部 6 个指标
@app.route("/api/kpi")
def get_kpi():
    province = request.args.get("province", "").strip()
    where, params = province_where(province)
    rows = query(f"""
        SELECT COUNT(*) AS station_count,
                COALESCE(SUM(fast_piles), 0) AS fast_piles,
                COALESCE(SUM(slow_piles), 0) AS slow_piles,
                COUNT(DISTINCT city) AS city_count,
                ROUND(AVG(electricity_price), 3) AS avg_electricity,
                ROUND(AVG(service_price), 3) AS avg_service
        FROM charging_stations{where}
    """, params)
    return jsonify({"code": 0, "data": dict(rows[0])})
```

```
def province_where(province):
    if province:
        return " WHERE province = ?", [province]
    return "", []
```

三、地图与下钻联动

地图用本地 china.json，注册成 ECharts 的 map 后，把 /api/map-data 返回的各省站点数当 value 着色。点击省份时调用 setFilterProvince()，它重新请求 5 个接口并覆盖渲染。这里有个细节：ECharts 地图是按 name 匹配行政区的，接口返回的字段是 province，需要在 JS 里补一个 name 映射，否则整张图上不了色。

```
// 节选自 app/static/js/screen_engine.js
const data = (res.data || []).map(d => Object.assign({}, d, { name: d.province }));

function setFilterProvince(provinceName) {
  currentProvince = provinceName || "";
  document.getElementById("activeFilterText").innerText = currentProvince || "全国视图";
  loadKpi();
  initScaleChart();
  initPileChart();
  initCityChart();
  initPriceChart();
  loadStationFeed();
}

// 1920×1080 固定画布按窗口等比缩放
function fitScreen() {
  const stage = document.getElementById("screen-stage");
  const s = Math.min(innerWidth / 1920, innerHeight / 1080);
  stage.style.transform =
    `translate(${(innerWidth - 1920 * s) / 2}px, ${(innerHeight - 1080 * s) / 2}px) scale(${s})`;
```

}

四、运行截图

图 1 是全国默认视图：33,652 座充电站、309,180 个快充桩、36,440 个慢充桩，覆盖 633 个城市；图 2 是点击广东省之后的下钻视图，指标卡变成 3,503 座站、38,483 个快充桩、2,950 个慢充桩，城市 TOP10 只剩广东的城市。两张图都是 Selenium 无头 Chrome 在 1920×1080 视口下截的。



图 1 全国默认视图：6 个指标卡 + 5 张图表 + 底部站点明细



图 2 广东省下钻视图：指标卡与 4 个图表同步切换，右上角出现重置入口

3、结果分析和总结

一、完成情况

题目要求基本都达到了：数据 33,652 条（要求 ≥1,000 条），有明确的公开来源；6 个聚合接口覆盖总量、空间分布、城市排名、规模结构、快慢充结构、价格分布 6 个维度（要求 ≥4）；页面 5 张 ECharts 图 + 6 个指标卡（要求 ≥5 图、≥3 卡），深色主题；点击地图能联动下钻（要求至少一种联动）；统计全部在 SQL 里完成，前端只渲染；页面按 1920×1080 设计，一屏完整显示。

数字核对：全国 33,652 条、快充 309,180 个、慢充 36,440 个、633 个城市；广东省 3,503 条、快充 38,483 个、慢充 2,950 个，和直接用 SQL 查的结果一致。

时间上从 10 月 10 日定题到 10 月 29 日完成数据整改和文档，中间经历了接口、前端、数据源三轮大改，M0~M5 六个里程碑都按时完成。

二、遇到的问题

1. 地图不上色。地图轮廓正常但所有省份一个颜色，查了半天发现 /api/map-data 返回的字段叫 province，ECharts 地图按 name 匹配，前端补了一层 name 映射才出图。

2. 带参数打开时联动失效。用 /?province=广东省 打开，右上角显示已下钻，但指标还是全国的——初始化的并发请求把下钻结果覆盖了。改成先 await 完成初始化，再应用 URL 里的省份参数。

3. 价格解析失败率 80%。原始数据有五种价格写法，第一版正则只认一种。补上“电价/服务费”标签、区间取中点、单值四种规则后，缺失率降到 5% 左右。

4. 电费出现 0 元和 5 元/度。个别记录把别的字段混进了充电费，按业务常识把 <=0 或 >3 的置空再用同省中位数补，处理了 889 条。

5. 端口被占用。调试过程中有几个旧 Python 进程一直占着 8080，新服务起来了但访问到的还是旧数据。用 netstat 找到 PID 结束进程后才正常。

6. 截图尺寸不对。只设 --window-size 截出来是 1902×928，底部表格被裁。改用 CDP 的 Emulation.setDeviceMetricsOverride 把视口锁成 1920×1080，同时页面改成固定画布缩放。

三、心得体会和不足

这次做下来最大的感受是“数据决定了大屏长什么样”。一开始用随机生成的数据，图表怎么调都像是假的：品牌份额、电池结构都太均匀，一眼能看出来。换成真实充电站数据之后，江苏、广东、山东的站点明显更多，城市排名、电费区间都有真实的分布特征，图才有说服力。

第二个体会是清洗比写接口费时间。价格字段的五种格式、地址里没有市名、0 元和 5 元

的异常值，这些问题不处理，后面的统计全是被污染的。把每一步的处理条数记下来（cleaning_report.json），写报告和答辩的时候心里有底。

不足的地方：一是数据只有站点静态信息，没有时间维度，所以做不了真正的趋势图，只能做分布类图表；二是城市解析还有约 4,800 条记录没有市名，只能归到“未知”；三是没写自动化测试，接口靠手工点；四是底部明细列表只是随机抽样，不是真正的实时数据流。

二、代码仓库链接与 AI 协作说明（必填）：

代码仓库链接（GitHub 或 Gitee，私有仓库需开放访问权限）：https://github.com/你的用户名/sk3017-screen-analytics（提交前替换为真实地址，并确认仓库可被老师访问）

AI 工具使用情况说明：本项目使用 Google Antigravity 辅助开发，主要用在三块——确定数据源与清洗规则、生成 Flask 接口和 ECharts 配置初稿、帮忙定位地图不上色和截图尺寸这类问题。生成的代码没有直接照抄，清洗规则、SQL 和联动逻辑都按自己的理解改过、调试过；关键提示词和采纳修改过程整理在仓库 docs/AI 协作记录.md。